SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	Pugazharasan K	**Reg. No.:**	192511341
Section / Batch:	B	**Date:**	28/08/2026

Code of Conduct

I Pugazharasan K (192511341) *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *Pugazh.K*

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
		GRAND TOTAL:	**100 Marks**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: ____________	Signature: ____________	Signature: ____________
Date: _______________	Date: _______________	Date: _______________

PSEUDOCODE WRITING TASK

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

a. All zones must be connected

b. Total installation cost must be minimized

c. No redundant connections (no cycles) allowed

d. The network must remain fully connected

Your Answer

```
1. MST ← empty set
2. Sort all edges of G in increasing order of weight
3. Create a separate set for each vertex
4. For each edge (u, v) in sorted edges:
     If FIND(u) ≠ FIND(v):
       Add edge (u, v) to MST
```

PSEUDOCODE WRITING TASK

Back

QUESTIONS

Q1
Minimum Spanning Tree - Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm - MST & Dijkstra's Algorithm
20 marks

Q4
Shortest Path Algorithm - Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree - Prim's or Kruskal's Algorithm
20 marks

Q2 **Graph - Topological Sort**

20 marks

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

a. Detect cycle before sorting

b. Use DFS approach

Your Answer

```
1. Create visited[V] and recursionStack[V]
2. Initialize all values of visited and recursionStack to FALSE
3. Create an empty stack
4. For each vertex v in G:
     If visited[v] = FALSE:
       If DFS(v) = TRUE:
```

Save Answer

Next

PSEUDOCODE WRITING TASK

Back

QUESTIONS

Q1
Minimum Spanning Tree - Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm - MST & Dijkstra's Algorithm
20 marks

Q4
Shortest Path Algorithm - Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree - Prim's or Kruskal's Algorithm
20 marks

Q3 **Shortest Path Algorithm - MST & Dijkstra's Algorithm**

20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.
Constraints:
a. Use MST for infrastructure
b. Use shortest path for routing

Your Answer

```
1. MST ← KRUSKAL(G)
2. Create distance[V]
3. Create visited[V]
4. For each vertex v:
     distance[v] ← ∞
     visited[v] ← FALSE
```

Save Answer

Next

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1
Minimum Spanning Tree - Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm - MST & Dijkstra's Algorithm
20 marks

Q4
Shortest Path Algorithm - Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree - Prim's or Kruskal's Algorithm
20 marks

Q4 **Shortest Path Algorithm - Dijkstra's Algorithm** 20 marks

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

```
1. For each vertex v:
     dist[v][0] ← ∞
     dist[v][1] ← ∞
2. dist[S][0] ← 0
3. Insert (0, S, 0) into Priority Queue
4. While Priority Queue is not empty:
```

Save Answer   Next →

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1
Minimum Spanning Tree - Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm - MST & Dijkstra's Algorithm
20 marks

Q4
Shortest Path Algorithm - Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree - Prim's or Kruskal's Algorithm
20 marks

Q5 **Minimum Spanning Tree - Prim's or Kruskal's Algorithm** 20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST. (20 Marks)
Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

```
1. MST ← KRUSKAL(G)
2. MST_Cost ← Total weight of MST
3. SecondBest ← ∞
4. For each edge e in MST:
    Remove e from MST
    Find the minimum-weight edge f
```

Save Answer